

Aprendizaje Multiagentes

Modelación de sistemas multiagentes con gráficas computacionales

Pedro Oscar Pérez Murueta, PhD

Tecnológico de Monterrey

✉ pperezm@tec.mx

🐙 github.com/Manchas2k4

Juego ficticio

Juego ficticio

- El juego ficticio es una de las primeras reglas de aprendizaje.
- De manera general, no es una regla de aprendizaje. Se podría considerar más un método iterativo para calcular el equilibrio de Nash.
- Es una instancia de aprendizaje basado en modelos, en donde el agente mantiene “creencias” acerca de la estrategia del oponente.

- La estructura de esta técnica es la siguiente:

Procedure 1 Juego Ficticio

Inicializar creencias acerca de la estrategia del oponente

while *NOT FINISHED* **do**

 Juega la mejor respuesta a la estrategia evaluada del oponente

 Observar el juego real de oponente y actualizar las creencias en consecuencia

end while

- Esta estrategia no considera las ganancias obtenidas (o que podrían obtener) los demás agentes. Se asume que cada agente conoce únicamente su propia matriz de ganancias y toma decisiones con base en la información disponible para sí mismo en cada paso.

Matriz de ganancia

- Una matriz de ganancias (o matriz de pagos) es una tabla que muestra las recompensas que recibe cada jugador según las decisiones tomadas por todos los participantes. Permite visualizar las consecuencias de cada combinación de estrategias y analizar qué decisiones resultan más convenientes para cada agente.
- Por ejemplo:
 - Dos jugadores tienen una moneda y de manera independiente cada uno de ellos elige mostrar Águila o Sol. Si ambos son iguales, el jugador 1 obtiene ambas. De otra forma, el jugador 2 consigue ambas.

Toma de Decisiones en Juego Ficticio

- En un inicio, los agentes usan una estrategia mixta dada la distribución empírica de las acciones previas del oponente.
- Si A es el conjunto de acciones del oponente, y por cada $a \in A$ definimos $w(a)$ como el número de veces que el oponente ha realizado la acción a . Entonces, el agente evalúa que la probabilidad de a en la estrategia mixta del oponente es:

$$Prob(a) = \frac{w(a)}{\sum_{a' \in A} w(a')}$$

- Por ejemplo, si el oponente juega Águila, Águila, Sol, Águila y Sol en las primeras rondas, antes de que se juegue la sexta ronda, se puede asumir que se jugará la estrategia mixta $(0,6, 0,4)$.
- La creencia de jugada de un oponente se puede representar como una medida de probabilidad, o como un conjunto de contadores $(w(a_1), w(a_2), \dots, w(a_k))$.

- Digamos que ...
 - Jugador 1 empieza el juego con la creencia de que Jugador 2 ha jugado Águila 1.5 veces y Sol 2 veces.
 - Jugador 2 empieza el juego con la creencia de que Jugador 1 ha jugado Águila 2 veces y Sol 1.5 veces.
- ¿Cómo jugarán?

Ronda	Acción J1	Acción J2	Creencia J1	Creencia J2
0			(1.5, 2)	(2, 1.5)
1	S	S	(1.5, 3)	(2, 2.5)
2	S	A	(2.5, 3)	(2, 3.5)
3	S	A	(3.5, 3)	(2, 4.5)
4	A	A	(4.5, 3)	(3, 4.5)
...	...	...	...	...

- En el juego anterior, cada uno de los jugadores termina alternando entre Águila y Sol.
- De hecho, si el juego tiende a infinito, la distribución de las jugadas de cada jugador convergerá a $(0,5, 0,5)$.
- Si se toma esta distribución de estrategias mixtas, el juego converge a un equilibrio de Nash, en donde cada jugador juega una estrategia mixta $(0,5, 0,5)$.

Caso de Estudio: Recolección de Energía

- Dos robots deben recolectar energía de dos estaciones, cada una capaz de generar 10 unidades de energía. Si ambos robots eligen la misma estación, deben compartir los recursos y cada uno recibe únicamente 5 unidades. En cambio, si cada robot selecciona una estación diferente, ambos obtienen las 10 unidades completas.

	Robot 2 al Norte	Robot 2 al Sur
Robot 1 al Norte	5	10
Robot 1 al Sur	8	4

Cuadro: Matriz de ganancia

- En este enfoque, el agente asume que la frecuencia pasada de las acciones del rival es una buena estimación de su comportamiento futuro.

- Por ejemplo, tras 20 iteraciones, el agente ha registrado que su oponente se ha desplazado hacia el norte 15 veces y hacia el sur 5 veces. A partir de esta información, puede estimar la frecuencia con la que cada acción es seleccionada y utilizarla para predecir el comportamiento futuro de su oponente.

$$P(Norte) = \frac{15}{20} = 0,75 \qquad P(Sur) = \frac{5}{20} = 0,25$$

- Con base en esta información, nuestro agente puede razonar de la siguiente manera: ‘ ‘Si decido ir hacia el norte, entonces...”

$$0,75 \times 5 + 0,25 \times 10 = 6,25$$

- ‘ ‘Y si decido desplazarme hacia el sur, entonces...”

$$0,75 \times 8 + 0,25 \times 4 = 7$$

- Por lo tanto, elige ir al sur.

Ver código en Python

Aprendizaje Racional

Aprendizaje Racional

- Aprendizaje racional (Rational Learning, también llamado Aprendizaje Bayesiano) adopta el mismo esquema basado en el modelo de Juego Ficticio.
- La principal diferencia con juego ficticio es que permite a los jugadores tener un conjunto de creencias más rico acerca de las estrategias de los oponentes.
 - Permite incluir estrategias de juegos repetidos.
 - Las creencias de cada jugador acerca de las estrategias del oponente pueden ser expresadas por cualquier distribución de probabilidad sobre el conjunto de todas las estrategias posibles.

- Al igual que la estrategia de Juego Ficticio, cada jugador empieza con alguna creencia.
- Después de cada ronda, el jugador usa una actualización Bayesiana para actualizar sus creencias.
- Sea S el conjunto de estrategias del oponente consideradas posibles por el jugador i , y H , el conjunto posible de historias en el juego (rondas pasadas). Entonces, la regla de Bayes para expresar la probabilidad asignada para jugador i al evento en donde el oponente está jugando una estrategia en particular $s \in S$ dada la observación histórica $h \in H$, es:

$$Prob(s|h) = \frac{Prob_i(h|s)Prob_i(s)}{\sum_{s' \in S} Prob_i(h|s')Prob_i(s')}$$

- Aprendizaje Racional es un modelo muy intuitivo para aprendizaje, pero su análisis es bastante complicado.
- El análisis se centra en el auto-juego, es decir, en las propiedades del juego repetido en el que todos los agentes emplean el aprendizaje racional.
- En términos generales, los aspectos más destacados de este modelo son los siguientes:
 - En algunas condiciones, en el auto-juego, el aprendizaje racional da como resultado que los agentes tengan creencias cercanas a las correctas sobre la parte observable de la estrategia de su oponente.
 - En algunas condiciones, en el auto-juego, el aprendizaje racional hace que los agentes converjan hacia un equilibrio de Nash con alta probabilidad.
 - La principal de estas “condiciones” es la continuidad absoluta, una fuerte suposición.

Retomando el caso de estudio

- En este enfoque, durante cada iteración, cada agente realiza las siguientes acciones:
 1. **Observa** las decisiones tomadas por los demás agentes.
 2. **Calcula** la frecuencia con la que cada acción ha sido seleccionada a lo largo de las interacciones previas.
 3. **Estima** que dichas frecuencias reflejan la estrategia que los demás agentes seguirán en el futuro.
 4. **Selecciona** la acción que constituye la mejor respuesta posible frente a la estrategia estimada de sus oponentes.
- De esta forma, los agentes van aprendiendo cómo es que pueden obtener una mejor recompensa.

Ver código en Python

Aprendizaje Reforzado

Aprendizaje Reforzado

- En Aprendizaje Reforzado (Reinforcement Learning), el agente interactúa con el mundo y periódicamente obtiene una recompensa que refleja que tan bien lo está haciendo.
- El agente no tiene idea de lo que tiene que hacer, solo realiza acciones y dependiendo si fueron correctas o no, recibe una recompensa (puntos por hacerlo bien, ningún punto si lo hace mal, o 1/2 si llegó hasta cierto punto).
- En ese sentido, el agente tiene que actuar más para conocer más. Imagina jugar un juego en donde no conoces las reglas; después de 100 movimientos o más, un arbitro te dice que perdiste. Eso es el aprendizaje reforzado.

- Este tipo de aprendizaje es útil cuando el agente tiene que trabajar en ambientes desconocidos usando solo sus percepciones y recompensas ocasionales.
- El diseño general para agentes define el tipo de información que debe ser aprendida:
 - Un agente basado en un modelo de aprendizaje reforzado obtiene (o se encuentra equipado con) un modelo de transición $Prob(s'|s, a)$ (donde a representa una evidencia de fondo) para el ambiente y aprende con función de utilidad $U(s)$ definido en términos de la suma de las recompensas del estado s hacia adelante.
 - Un agente libre es un modelo de aprendizaje pudiera aprender una función de acción-utilidad $Q(s, a)$ o una política $\pi(s)$.

Función de acción-utilidad $Q(s, a)$

- El agente aprende una función Q , o función de calidad, $Q(s, a)$ que es la suma de las recompensas del estado s en adelante si la acción a es realizada.
- Dada una función Q , el agente puede elegir que hacer en s al encontrar la acción con el valor de Q más alto.
- De forma general, Q se representa como una tabla de valores de acciones indexada por estado y acción, inicialmente en 0's. Y V es una tabla de frecuencias estado-acción, inicialmente en 0.

El cálculo de la función de calidad es la siguiente:

$$Q_{t+1}[s_t, a_t] = (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{valor viejo}} + \underbrace{\alpha}_{\text{tasa de aprendizaje}} \cdot \overbrace{\left(\underbrace{r(s_t, a_t)}_{\text{recompensa}} + \underbrace{\gamma}_{\text{factor de descuento}} \cdot \max_a Q(s_{t+1}, a_t) \right)}^{\text{valor aprendido}}$$

donde:

- $r(s_t, a_t)$ es la recompensa recibida al pasar del estado s_t al estado s_{t+1} .
- α es la tasa de aprendizaje ($0 < \alpha < 1$).
- γ es un factor de descuento ($0 < \gamma < 1$) y evalúa las recompensas recibidas anteriormente con un valor mayor que las recibidas posteriormente.

- α determina hasta qué punto la información adquirida sobrescribe la información vieja.
- Un factor de 0 hace que el agente no aprenda (únicamente aprovechando el conocimiento previo), mientras un factor de 1 hace que el agente considere sólo la información más reciente (ignorando el conocimiento previo para explorar posibilidades).
- En entornos totalmente deterministas, un índice de aprendizaje de $\alpha_t = 1$ es óptimo. $\alpha_t = 1$ cuando el problema es estocástico, el algoritmo converge bajo determinadas condiciones técnicas en el índice de aprendizaje que requiere un descenso hasta cero.
- En la práctica, a menudo se utiliza un índice de aprendizaje constante, como $\alpha_t = 0,1$ para toda t .

- γ determina la importancia de las recompensas futuras.
- Un factor de 0 hará que el agente sea “miope” por considerar únicamente las recompensas actuales.
- Mientras que un factor que se acerca a 1 hará que luche por una recompensa alta a largo plazo.

El algoritmo que implementa la regla de actualización anterior se define a continuación:

Procedure 2 Aprendizaje Q

Inicializar la función Q y V valores.

while $\neg$ converge **do**

 Observar el estado actual s_t

 Seleccionar acción a_t

 Observar la recompensa $r(s_t, a_t)$

$Q_{t+1}[s_t, a_t] \leftarrow (1 - \alpha) \cdot Q(s_t, a_t) + \alpha \cdot (r(s_t, a_t) + \gamma \cdot \max_a Q(s_{t+1}, a_t))$

$V[s_t, a_t] \leftarrow \max_a Q(s_{t+1}, a_t)$

end while

Políticas

- El comportamiento de un agente se encuentra modelada como un mapa llamado política:

$$\begin{aligned}\pi : A \times S &\rightarrow [0, 1] \\ \pi(a, s) &\leftarrow \text{Prob}(a_t = a | s_t = s)\end{aligned}$$

- El mapa de la política provee la probabilidad de tomar la acción a cuando se está en el estado s .
- En este método, la idea es ir actualizando las políticas siempre y cuando el desempeño mejora.

- Una de las ventajas de aprendizaje reforzado es que no es necesario la construcción manual de los comportamientos y del etiquetado de un vasto conjunto de referencias usando en aprendizaje supervisado o definir a mano las secuencias de control de las estrategias.
- Aprendizaje reforzado es útil cuando nos encontramos en un ambiente donde es necesario manejar entornos continuos, o ambientes con alta dimensionalidad o parcialmente observable en donde comportamientos exitosos pueden consistir en miles o millones de acciones.

Retomando el caso de estudio

- Cada robot mantiene una tabla de valores (Q-table) que almacena la recompensa esperada asociada a cada acción disponible:
 - **Q(Norte)**: recompensa esperada al desplazarse hacia el norte.
 - **Q(Sur)**: recompensa esperada al desplazarse hacia el sur.
- Inicialmente, el robot no posee información sobre cuál de las dos acciones es más conveniente, por lo que asigna el mismo valor a ambas:
 - **Q(Norte) = 0**
 - **Q(Sur) = 0**
- Durante cada ronda, el robot selecciona una acción siguiendo una de dos estrategias:
 - **Exploración**: prueba una acción de manera aleatoria para obtener nueva información.
 - **Explotación**: elige la acción que actualmente posee el mayor valor Q.

- Supongamos que, después de varias interacciones, el robot tiene la siguiente estimación:
 - **$Q(\text{Norte}) = 6$**
- Si en la siguiente ronda decide desplazarse hacia el norte y obtiene una recompensa de:
 - **Recompensa = 10**entonces actualiza el valor asociado a esa acción utilizando la recompensa observada. Como resultado, la estimación de $Q(\text{Norte})$ aumenta, reflejando que esta acción parece ser más beneficiosa de lo que se creía inicialmente.
- De esta manera, el robot mejora progresivamente sus estimaciones y aprende cuáles acciones producen mayores recompensas a lo largo del tiempo.

Ver código en Python

¿Quieres saber más?

- Capítulos 12 y 22 de libro de Russel y Norvig.
- Capítulo 7 del libro de Shoham y Leyton.